NEELEY AI FORWARD | HOW WE DID IT

The Build: how Pharmer reads the fine print

The technical part of our team's report.

Tools and what each was for

We built a reusable Python pipeline, **Pharmer**, using **Claude Code** (Opus 5.5) as our coding environment and **Codex** to independently review the codebase. Inside the pipeline, **Claude Opus 5.5** does the judgement work: six "hunter" agents that look for findings, the product comparison, the verdict and the CEO move. **Claude Haiku 5.5** does the mechanical checks: merging duplicate findings, judging whether a quote supports its claim, and choosing a formula to size a finding. Filings and figures come from **SEC EDGAR** and its XBRL data, labels from **DailyMed** and FX rates from the **ECB**. **Streamlit** runs the UI, and **reportlab** renders the PDF.

Workflow

No documents were pasted into a chat. Code fetches each filing, splits it into chunks tagged with the filing's printed page number, and the agents search and read those chunks through tools. A finding is rejected unless its quote matches the filing word for word. Partner A built the data and checking stages, and Partner B built the agents and the output. Shared data contracts and an append-only build log kept two separate AI sessions consistent. Humans chose the company, the Sharpest Needle, the product and every scope cut. Outputs are checked in code, not by asking the model. Every number is tied to a second source (XBRL), Python does all the arithmetic, the AI judge can only lower a finding's confidence, and every number in the verdict or CEO move must appear in that call's input. The code itself passed 503 offline tests and several multi-reviewer code reviews. A full live run on Eli Lilly completed.

Most important prompts

Three rules carried most of the weight, and each is also enforced in code. For every hunter: *"Never use facts, numbers, dates, quotes or events from your own memory or training data, even if you are sure they are true."* For sizing a finding: *"You do not compute the answer: Python evaluates your formula against the inputs you name."* For the CEO move: *"It must be a decision, not more research: never 'investigate', 'monitor', ..."*

Where the AI was confidently wrong

On the first live run, the sizing stage returned a **positive** number for every write-down, so a loss would have printed as a gain. We caught it by reading the output rather than trusting that the stage finished cleanly. The model no longer sets the sign: it must label each amount a cost or a benefit, and Python applies the sign. Earlier, a web-search summary stated that Novo's Q2 sales grew **+7%** at constant exchange rates, and the 6-K showed **+3%** reported. We caught it by opening the filing. The same confusion later came back through our own code, which matched the table row "Net sales at CER" as revenue. An independent reviewer caught that one. In total we logged 33 such cases, almost all caught by going back to the primary source or by a second reviewer.

What we would do differently

First, verify the API key on day one. We lost hours to key errors and built most stages against a fake client before any real call worked. Second, fix the sizing stage's assumed rates in a reviewed table: two runs on identical Lilly evidence chose different write-down rates, and the recommended finding flipped. Third, load earnings-call transcripts and peer filings earlier, so the tool can measure whether a finding is already priced in, instead of leaving that judgement entirely to the analyst.